

Universidade de Aveiro

Departamento de Eletrónica, Telecomunicações e Informática

API UA

Technical Report

PECI

Equipa

André Reis: 119814

Vasco Oliveira: 119113

Pedro Ferreira: 119240

Carlos Ramos: 119873

Dinis Malhado: 120420

Simão Pereira: 120459

Orientador: Diogo Gomes

June 4, 2026

Abstract

For over a decade, the `api.web.ua.pt` platform served as a vital digital backbone for the University of Aveiro, centralizing essential services ranging from canteen menus to academic ticketing. However, as the digital landscape evolved, the original infrastructure began to show its age, struggling with outdated security protocols, poor readability, and a lack of modern architectural standards. These mounting technical limitations eventually led to the decommissioning of the legacy system, sparking the urgent need for a complete, from-the-ground-up rebuild to better serve the modern student body. This project outlines the comprehensive modernization of the university's API ecosystem. The reconstruction began by revitalizing core endpoints—including canteen services (`sas`), RSS-to-JSON translation tools (`Rss2Json`), institutional data (`ArcGIS`), real-time parking availability (`parques`), and academic services (`sac`). By employing current programming languages and libraries, the fundamental bugs and inefficiencies of the previous system were addressed. To safeguard these assets against external threats, the new APIs were integrated into a robust WSO2 management layer, unified under Keycloak as the platform's identity provider. The result is a secure, high-performance platform complemented by a dedicated documentation portal and an interactive testing framework, ensuring that the university's digital tools are as reliable and accessible as the services they represent.

Contents

Abstract	i
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.3 Goals and Objectives	2
1.4 Document Structure	2
2 Related Work, Tools and Technologies	3
2.1 Related Work	3
2.1.1 Commercial API Management Platforms	4
2.1.2 Comparable Institutional Platforms	4
2.2 Tools and Technologies	5
2.2.1 Python and FastAPI	5
2.2.2 WSO2 API Manager	5
2.2.3 Keycloak	5
2.2.4 Docusaurus and Stoplight Elements	6
2.2.5 Memcached	6
2.2.6 Grafana, Loki and Promtail	6
2.2.7 Docker	6
3 Requirements and Architecture Overview	7
3.1 Requirements Elicitation	7
3.1.1 Functional Requirements	7
3.1.2 Non-Functional Requirements	8
3.2 Architecture Overview	8
3.2.1 User Interface — Academic Playground & Innovation	9
3.2.2 WSO2 API Gateway	9
3.2.3 FastAPI Backend — ServicesUAPT	9
3.2.4 Backend Data Sources	9
3.2.5 Monitoring	9

4	Implementation	10
4.1	Local Development Environment	10
4.2	WSO2 API Manager	10
4.2.1	Configuration	10
4.2.2	API Registration and Lifecycle	10
4.2.3	Access Control and Scopes	10
4.3	Keycloak	10
4.3.1	Realm Configuration	10
4.3.2	User Roles and Clients	10
4.3.3	Integration with WSO2	10
4.4	APIs — ServicesUAPT	10
4.4.1	SAS — Canteen Menus	10
4.4.2	Parques — Parking Availability	10
4.4.3	Rss2Json	10
4.4.4	Access Points — Mock API	10
4.5	Academic Playground & Innovation — Documentation Portal	12
4.5.1	Structure and Content	12
4.5.2	Interactive Testing	12
4.5.3	Keycloak Integration	12
4.6	Monitoring — Grafana, Loki and Promtail	12
4.6.1	Log Collection with Promtail and Loki	12
4.6.2	Grafana Dashboard	12
5	Results	13
5.1	Expected Results	13
5.2	Achieved Results	14
5.2.1	APIs Delivered	14
5.2.2	Security and Access Control	14
5.2.3	Performance	14
5.2.4	Known Limitations and Future Work	14
5.3	Projected Schedule	14

List of Figures

3.1	System Architecture Overview	8
5.1	Projected Schedule	14

List of Tables

Chapter 1

Introduction

1.1 Context and Motivation

The University of Aveiro has long relied on a centralised web services platform — hosted at `api.web.ua.pt` — to expose institutional data to students, faculty, and third-party developers. Through this platform, the university made available a range of services that became embedded in the daily life of its academic community: real-time canteen menus, academic services queue tickets, campus building information, and parking availability, among others.

Despite its relevance, the platform was built over a decade ago on a PHP stack that has since reached end-of-life. Over time, the absence of security updates, the accumulation of unresolved bugs, and the growing incompatibility with modern development practices rendered the system increasingly difficult to maintain. These factors ultimately led to the decommissioning of the legacy platform, leaving a gap in the university’s digital infrastructure that directly affected students and developers who depended on it.

1.2 Problem Statement

The core issues that led to the platform’s decommissioning can be grouped into three categories.

The first is **security**. The legacy system ran on an outdated version of PHP with known CVEs that had gone unpatched, exposing the platform to exploitation. With no authentication or authorisation layer in place, any consumer could call any endpoint without restriction, making it impossible to protect sensitive data or audit access.

The second is **reliability and performance**. The architecture was not designed to handle concurrent load gracefully. Under peak usage — such as at the start of the academic year or during lunch hours — the server would become unresponsive,

degrading the experience for all users simultaneously.

The third is **maintainability**. The codebase lacked modularity and documentation, making it difficult to onboard new developers or extend the platform with additional services. Adding a new API required modifying a tightly coupled codebase with no clear separation of concerns.

1.3 Goals and Objectives

This project aims to deliver a complete, production-ready replacement for the legacy platform. The main objectives are:

- Migrate all existing services to a modern Python stack using FastAPI, preserving the original endpoints to ensure backward compatibility with existing consumers;
- Introduce a WSO2 API Gateway layer to centralise authentication, authorisation, monitoring, and rate limiting across all services;
- Develop a documentation and interactive testing portal — the *Academic Playground & Innovation* — that makes it straightforward for students and developers to discover, understand, and integrate the available APIs;
- Design the architecture to be modular and extensible, so that new services can be added with minimal effort and no disruption to existing ones.

1.4 Document Structure

This report is organised into five chapters. Chapter 2 surveys existing solutions and API gateway platforms, and presents the tools and technologies selected for the project, justifying each choice. Chapter 3 covers the requirements elicitation process and provides a high-level description of the system architecture. Chapter 4 details the implementation of each architectural component, including configuration, local development environment, and integration between services. Chapter 5 outlines the expected outcomes and evaluates the results achieved against the initial objectives.

Chapter 2

Related Work, Tools and Technologies

2.1 Related Work

Two previous projects developed at the University of Aveiro are particularly relevant to this work: the *OPEN DATA UA* master thesis and the final report of the *Academic Playground & Innovation (API)* project. Together, they provide both the conceptual and practical foundations on which this project is built.

The *OPEN DATA UA* thesis surveys existing institutional information sources and web services at UA and proposes a service-oriented architecture based on REST and SOAP for exposing institutional data (e.g., canteen menus, academic information) in a standardized way. It also implements a web portal (APIIn) that aggregates services from UA, PT Inovação and SAPO, and introduces an OAuth-based authorization server to protect access to private or sensitive web services. This work is mainly focused on identifying data sources, creating individual web services, and demonstrating how open data and secure access control can be applied within the university context.

The *Academic Playground & Innovation* final report documents the concrete implementation of that vision, namely the `api.web.ua.pt` portal and the supporting services running on `services.web.ua.pt` and `identity.ua.pt`. It describes in detail the API portal, the catalogue of services (UA, PT SAPO, PT Inovação), the documentation and wiki mechanisms, the statistics and administration interfaces, as well as the deployment of an OAuth 1.0a server integrated with the UA Identity Provider using SimpleSAMLphp. This report provides an operational view of how service publication, authentication, authorization and consumption were handled in practice in the first generation of the institutional API platform.

This project can be seen as a next step in this evolution. While *OPEN DATA UA* and the API portal focused on exposing individual services and providing a web-based aggregation and documentation layer, they are tightly coupled to specific technologies and infrastructures that have since become obsolete. In contrast, this project aims

at designing a modern institutional API Gateway and governance model, aligned with current API management practices. Building on the lessons learned from these previous projects, the goal is to provide a more sustainable, secure and governable integration layer for present and future UA systems.

2.1.1 Commercial API Management Platforms

Several commercial and open-source API management platforms address problems similar to those this project aims to solve. **Kong Gateway** is a widely adopted open-source API gateway, offering plugins for authentication, rate limiting, logging, and traffic routing. It is highly performant and extensible, but requires significant infrastructure investment and operational expertise to deploy and maintain at scale. Given that one of the core goals of this project is long-term sustainability and ease of handover to future development teams, introducing a platform with such operational complexity would be counterproductive.

AWS API Gateway and **Azure API Management** are fully managed cloud offerings that eliminate the operational burden of running gateway infrastructure. Both provide built-in support for OpenAPI specifications, OAuth2 integration, and developer portals. However, as hosted proprietary services, they introduce vendor lock-in and are unsuitable for an institutional deployment that must remain under the university's control and on-premises.

RapidAPI represents a different model, functioning primarily as a marketplace and developer portal for discovering and testing third-party APIs. Its interactive testing interface — which allows developers to issue live requests directly from the documentation page — was a direct source of inspiration for the *Academic Playground & Innovation* portal developed in this project.

2.1.2 Comparable Institutional Platforms

Beyond commercial offerings, several universities and public institutions have developed API platforms to expose institutional data to their communities. MIT's *Atlas* and Stanford's developer portals follow a similar model: centralised access to institutional data through authenticated APIs, with developer documentation and sandbox environments.

The common thread across these platforms is the separation between the *gateway layer* — responsible for security, routing, and governance — and the *backend services* that implement the actual business logic. This separation is a key architectural principle adopted in this project, and one that was notably absent from the legacy system, where security concerns, business logic, and data access were all handled within the same tightly coupled PHP codebase with no clear boundaries between layers.

2.2 Tools and Technologies

2.2.1 Python and FastAPI

Python was selected as the primary implementation language for the backend services. Its extensive ecosystem, readability, and widespread adoption in both academia and industry make it a natural fit for a platform that must be maintained by successive generations of students. FastAPI is a modern Python web framework designed for building APIs with high performance and minimal boilerplate. It leverages Python type annotations to generate OpenAPI-compliant documentation automatically at runtime, eliminating the need to maintain separate API specification files. This tight integration between code and documentation is particularly valuable in a context where the OpenAPI specification is consumed directly by both the WSO2 gateway and the developer portal.

2.2.2 WSO2 API Manager

WSO2 API Manager is an open-source, on-premises API management platform that provides a complete solution for publishing, securing, monitoring, and governing APIs. It was selected for this project because it supports the full OAuth2 authorization code flow natively, integrates with external identity providers via OpenID Connect — enabling seamless federation with the Keycloak instance — and exposes a standards-based management interface that can ingest OpenAPI specifications directly. Its open-source nature and on-premises deployment model ensure that the university retains full control over the platform without dependency on third-party cloud providers.

2.2.3 Keycloak

Authentication and authorisation are handled through the OAuth2 protocol, delegated to a **Keycloak** instance configured with a dedicated `ua` realm. Keycloak was chosen for its mature support for OAuth2 and OpenID Connect, its flexible role-based access control, and its ability to federate with external identity providers. The WSO2 gateway validates tokens issued by Keycloak before forwarding requests to the backend, and access scopes are mapped to user roles — `admin`, `professor`, and `aluno` — allowing fine-grained control over which endpoints each profile can consume. The Docusaurus developer portal is also integrated with Keycloak, enabling users to authenticate directly from the documentation interface.

2.2.4 Docusaurus and Stoplight Elements

The developer portal is built with **Docusaurus**, a React-based static site generator maintained by Meta. It was chosen for its native support for Markdown and MDX content, built-in versioning, and low maintenance overhead — documentation updates require only editing text files, with no application code changes. Interactive API testing is provided by **Stoplight Elements**, an open-source React component that renders a full API explorer from an OpenAPI specification. All test requests issued through the portal are routed through the WSO2 gateway, ensuring that the developer experience accurately reflects production behaviour.

2.2.5 Memcached

Memcached is a high-performance, in-memory key-value store used in this project to cache upstream API responses. Given that certain data sources — such as the UA WSO2 canteen menu endpoint — are slow to respond and return data that changes at most once per day, caching responses eliminates redundant upstream calls and significantly reduces response times for end consumers. Memcached was preferred over alternatives such as Redis for its simplicity and minimal resource footprint, which is particularly relevant given the constrained hardware available for deployment.

2.2.6 Grafana, Loki and Promtail

Platform observability is provided by **Grafana** and **Loki**. Loki collects and indexes logs produced by the WSO2 API Manager, while Grafana provides a dashboard layer for visualising API usage metrics, error rates, and traffic patterns per endpoint. Log ingestion is handled by **Promtail**, which ships WSO2 logs to Loki in real time. A pre-provisioned dashboard is included in the repository, giving operators an immediate view of platform health without manual configuration.

2.2.7 Docker

All platform components are containerised using **Docker** and orchestrated via **Docker Compose**. The services run on a private Docker network (**gateway-net**), isolating them from external access. WSO2 API Manager is the only component exposed externally, acting as the single entry point for all API traffic.

Chapter 3

Requirements and Architecture Overview

3.1 Requirements Elicitation

The requirements for this project were gathered through direct engagement with the primary stakeholders. Meetings with STIC (Serviços de Tecnologias de Informação e Comunicação) and the project supervisor, Diogo Gomes, provided the functional and non-functional constraints that shaped the architecture.

3.1.1 Functional Requirements

- **FR1 – Service Centralisation and Exposure:** A WSO2 layer must exist that acts as a centralised API Gateway to expose the University of Aveiro’s services uniformly.
- **FR2 – Identity and Permissions:** The system must implement authentication and authorisation mechanisms, ensuring access control per API according to each user’s profile.
- **FR3 – Control and Monitoring:** The platform must allow detailed control and monitoring of all service accesses, including logging and traceability of calls made.
- **FR4 – Documentation Availability:** Documentation must be available for the use of each API, including description of endpoints, parameters, authentication, and usage examples.
- **FR5 – Continuity and Compatibility:** The platform must maintain the same endpoints, ensuring compatibility with already integrated systems and avoiding service interruptions.

3.1.2 Non-Functional Requirements

- **NFR1 – Security:** The system must comply with security best practices, mitigating vulnerabilities identified in the previous solution.
- **NFR2 – Performance:** API requests must be processed in less than 3 seconds under normal operating conditions.
- **NFR3 – Technical Scalability:** The API Gateway must support increased load and growth in the number of consumers without significant service degradation.
- **NFR4 – Functional Scalability:** The introduction and integration of new APIs must be simple, quick, and have minimal impact on existing services.
- **NFR5 – Maintainability:** The system must adopt software development best practices and keep code readable, modular, and well-documented.
- **NFR6 – Interface Usability:** The documentation portal must present a simple, clear, and intuitive interface, promoting adoption of the available services.

3.2 Architecture Overview

The platform is organised into four distinct layers, as illustrated in Figure 3.1: the *User Interface*, the *WSO2 API Gateway*, the *FastAPI Backend*, and the *Backend Data Sources*. All components run on a private Docker network (`gateway-net`), with WSO2 as the sole externally exposed entry point.

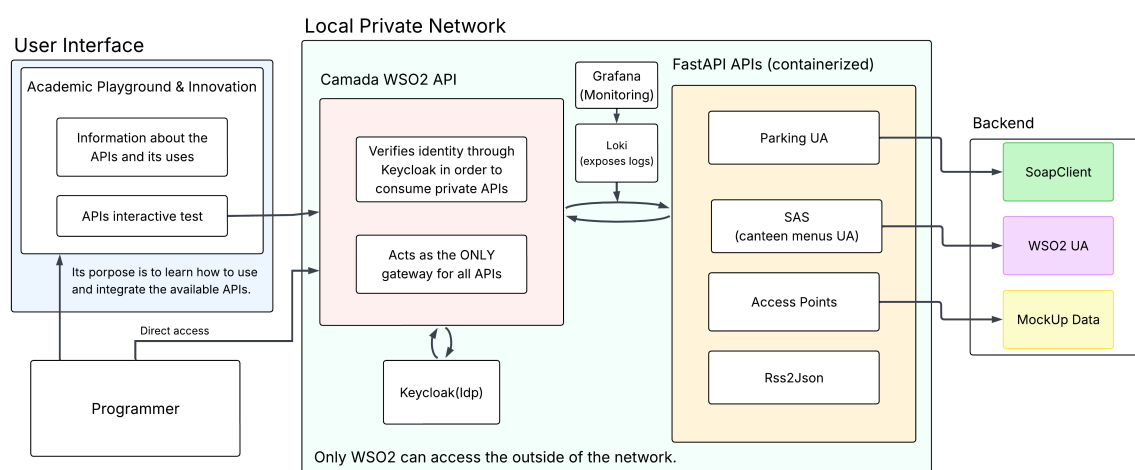


Figure 3.1: System Architecture Overview

3.2.1 User Interface — Academic Playground & Innovation**3.2.2 WSO2 API Gateway****3.2.3 FastAPI Backend — ServicesUAPT****3.2.4 Backend Data Sources****3.2.5 Monitoring**

Chapter 4

Implementation

4.1 Local Development Environment

4.2 WSO2 API Manager

4.2.1 Configuration

4.2.2 API Registration and Lifecycle

4.2.3 Access Control and Scopes

4.3 Keycloak

4.3.1 Realm Configuration

4.3.2 User Roles and Clients

4.3.3 Integration with WSO2

4.4 APIs — ServicesUAPT

4.4.1 SAS — Canteen Menus

4.4.2 Parques — Parking Availability

4.4.3 Rss2Json

4.4.4 Access Points — Mock API

The API UA platform is not merely a system that centralizes the various services offered by the University of Aveiro. Rather, it is designed to facilitate easy integration

of new APIs that may be developed in the future. To demonstrate this extensibility, we created a mock API to simulate the integration of new web services into the platform.

Our supervisor recommended developing a mock API for the University of Aveiro's access points, containing useful information such as their geographical locations, maximum connection capacity, and other data relevant to the academic community.

To store the mock API data we created a MySQL database hosted on the IETTA infrastructure. This decision was driven by the fact that, concurrently with this project, we attended a relational databases course that provided access to the IETTA database for class exercises and the course project. Given that existing access and the operational convenience it provided, we obtained permission from the course coordinator to use the IETTA database for persisting the data for our mock API.

The database schema contains two primary tables: one for access points and another for the departments where each access point is located. Each table includes several attributes, which are represented in the entity–relationship diagram (ERD) shown in Figure (??). The ERD was designed to organise and document the database structure.

Mock API Implementation Stack

The mock API was implemented in Python using a Flask application running on a virtual environment to isolate project dependencies and ensure a controlled, reproducible development environment without conflicts with other system installations. The application architecture relies on several key technologies:

- **Uvicorn** — the ASGI server responsible for running the application and exposing the API through HTTP requests;
- **PyODBC** — the library used to establish the connection between the Python application and the SQL Server database, using the ODBC protocol;
- **ODBC** — a standard that enables access to different database management systems in a uniform and language-independent way, providing database abstraction;
- **Virtual Environment** — isolates the project's dependencies and ensures reproducibility without conflicts with other system installations.

The overall architecture diagram is shown in Figure (??), illustrating the integration of these components within the broader API UA platform.

4.5 Academic Playground & Innovation — Documentation Portal

API spec pointing to WSO2

4.5.1 Structure and Content

4.5.2 Interactive Testing

4.5.3 Keycloak Integration

4.6 Monitoring — Grafana, Loki and Promtail

4.6.1 Log Collection with Promtail and Loki

4.6.2 Grafana Dashboard

Chapter 5

Results

5.1 Expected Results

At the end of this project the expected result is to have a fully operational platform that operates in accordance with the objectives that were proposed. The system will be completely secured, operating with Keycloak as the identity provider, to prevent any unwanted access to the private APIs implemented. The platform should also be intuitive and easy to use by students, professors and external developers alike that aim to integrate any of the available APIs into their own projects. To support this, a comprehensive documentation website will be created, covering each endpoint and method available, alongside an interactive testing mechanism that allows users to try all those methods directly in the browser. Due to the modernization and different implementations, such as the usage of Memcached caching and other technologies, responses are expected to be processed in under 3 seconds under normal operating conditions. Finally, all the work done will remain compatible with systems already integrated with the previous platform, as the endpoints will stay the same, avoiding service disruption and ensuring continuity for existing consumers.

5.2 Achieved Results

- 5.2.1 APIs Delivered
- 5.2.2 Security and Access Control
- 5.2.3 Performance
- 5.2.4 Known Limitations and Future Work

5.3 Projected Schedule

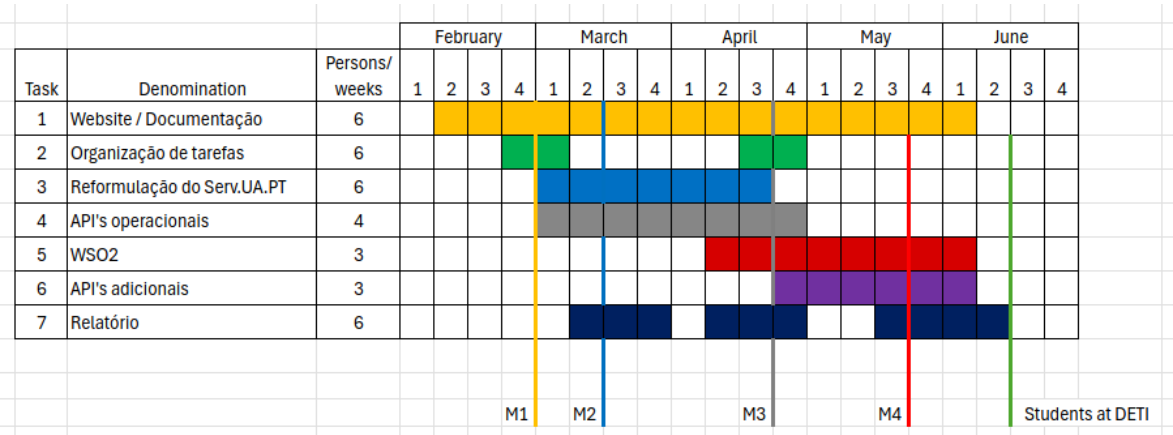


Figure 5.1: Projected Schedule

The planned work is organised into seven main tasks, distributed between February and June. Website and documentation activities span almost the entire period, providing continuous support to the remaining tasks. Initial weeks focus on task organisation and the refactoring of existing services, followed by the implementation and stabilisation of the core institutional APIs. In the final phase, additional APIs are integrated and the project report is completed, ensuring that the technical work is accompanied by up-to-date documentation and validation milestones.

Bibliography